

Nombre : HOOP ( Human Object Oriented Programming )

Palabras reservadas :

mold ( class), bond ( interface), listing ( enum), layout ( struct ), unit ( package ), if , else, cycle ( for ), while, stop ( break ), skip ( continue ), answer ( return ), choose ( switch ), create ( new ), self ( this), empty ( null ), parent ( super ), open ( public ), closed ( private ), linked ( protected ), handle ( try), catch, finally, raise ( throw), const, data ( var), action ( fun ), show ( print, printf)

Sintaxis :

### **Data declaration**

```
const <name>: <dataType> = <value> || empty;
data <name> : <dataType> = <value> || empty;
```

### **Struct**

```
layout <name> {
    open data <name>: dataType;
}

data <name> : <layoutName> = create <layoutName>;
<dataName>.<layoutData> = <value>;

show "<value>";
```

### **Class**

```
mold <name> {
    <access modifier> data <name>: <dataType> = <value>;
    // constructor
    action setup {}
    // destructor
    action destroyer {}

    action <name> ( <parameterName>: <dataType> ) {
        self.<attributeName>
    }

    // actionType is the same as simple dataTypes
    action <name>() : <actionType> {
        answer self.<returnValue>
    }
}
```

```

    }

}

data <name> = create <moldName>();
<objectName>.setup();
<objectName>.<actionName>(<parameterValue>);

```

### **Loops and conditions declaration**

```

data <name> : <dataType> = <value>;

if ( <dataName> > <value>) {
    show "<value>";
} else if (<dataName> < <value>) {
    show "<value>";
} else {
    show "<value>";
}

// cycle sintaxis
cycle <data> i form <value> to <value> {
    show i;
}

// list cycle
cycle <name> from <chain> {
    show <name>;
    if ( name = "<value>"){
        stop;
    }
}

```

//while sintaxis

```

while (condition) {
    show "<value>";
}

```

//choose sintaxis

```

data <name> : <dataType> = <value>;

choose <condition> {

```

```

    case 1 :
        show "<value>";
        stop;
    default :
        show "<value>";
}

```

## Error Handling

```

handle {

<operation>;

} catch {
    show "error";
} finally {
    show "<value>";
}

```

## Compound Data Types

```

// chain

data <name>: Chain<dataType> = create Chain<dataType>();
<chainName>.add("<value>");

// grid
data <name>: Grid<dataType> =create Grid<dataType>();
<gridName>.set(<positionX>,<positionY>,<value>);

```

Semántica :

HOOP cuenta con una semántica que asegura no sólo un código que sea sintácticamente correcto, si no también que se comporte de tal manera que cualquier persona pueda utilizarlo, siendo así un estilo de pseudocódigo, pero sin dejar de ser código. Hemos realizado énfasis en una ejecución predecible, con una clara versión del paradigma de orientado a objetos, con una versión comprensible del manejo de errores, todo eso mientras proveemos el poder del paradigma imperativo y el paradigma de orientado a objetos.

## 1. Ejecución predecible y controlada ( Paradigma imperativo)

HOOP cuenta con el paradigma imperativo, asegurando una naturaleza en las acciones realizadas paso a paso, en una secuencia, paso a paso, haciendo esto una manera sencilla de seguir lo que pasa en el código. Realizando las ejecuciones de manera secuencial, haciéndolo de manera que el código se ejecutará de principio a fin, dado por el bloque de “action”. Cuenta con un control de flujo explícito y con esto nos referimos a sus ciclos, como el if/else, cycle, while y choose. Dando no solo palabras entendibles a los usuarios para entender de qué hacen los ciclos si no lo sencillo que es seguir estas instrucciones dadas a la máquina, incluimos, las palabras stop, para poder detener el ciclo y la palabra skip para continuar con la ejecución. La declaración de variables es fácil de distinguir entre sus constantes y sus variables, declaradas las constantes por medio de la palabra reservada const y la palabra data para una variable típica. Además utilizamos la palabra mold, traducida al español como molde, para las clases, unit para las unidades o paquetes del código, action para las funciones, además de los ciclos que son fáciles de inferir. Incluimos aquí la declaración de variables y sus tipos, los cuales son fáciles de entender para cualquier persona, además de sus tipos de variables compuestos los cuales son pensados en la forma de cómo una persona lo puede imaginar, ya que como anteriormente se mencionó, este es un lenguaje donde el entendimiento del usuario es la prioridad.

## 2. Interacción intuitiva con los objetos ( Paradigma Orientado a Objetos)

Como anteriormente se mencionó, este lenguaje está diseñado para el entendimiento natural de las personas, por lo tanto se utilizan las palabras como open, closed y linked para las restricciones de acceso a los atributos de las clases o estructuras. Contamos con el polimorfismo y herencia, haciendo uso de las palabras extends, parent, implements. Dichas palabras son conocidas y entendibles para el usuario desarrollador. Además del ciclo de vida de los objetos, dando al usuario una forma más sencilla y entendible de poder crear un objeto haciendo el uso de la palabra create.

## 3. Manejo claro de errores

Con HOOP los errores son tratados no como crashes, si no como situaciones que pueden pasar y ser manejadas antes de que pasen, por lo tanto nuestro manejo de errores, es comprensible para el usuario, dándole una manera más simple de entender que está manejando, lanzando sus errores por medio de un raise, también haciendo uso de las palabras reservadas handle/catch y finally. Manejamos errores como un try, catch, ya que si algo falla en lo que debemos realizar lo agarramos para que nuestro sistema no se caiga, y podemos controlar nuestra aplicación

#### 4. Sistema de semántica legible para el humano

En HOOP encontramos una semántica amigable para el usuario, asegurando confiabilidad y seguridad. Asegurando que nuestras operaciones sobre de datos sean lógicas para el usuario desarrollador, que sea como escribir un mensaje de texto a un amigo. Por eso utilizamos un tipado fuerte en nuestro lenguaje, donde cada dato o constante debe de ser definido por su tipo de datos. De esta manera nos aseguramos que cada operación sea relacionada según el tipo de datos que tengamos. De esta manera reducimos sorpresas durante la ejecución del programa ya que nos aseguramos de que todo sea correcto antes del tiempo de compilación. Controlamos que no puedas realizar operaciones de suma entre un text y un whole, así con las demás operaciones aritméticas o lógicas. Hacemos uso de la palabra empty para aquellas variables que van a estar vacías sin importar el tipo, por lo que no es válido definir una variable sin un valor, y si es el caso de que lo necesites, vas a tener que usar nuestra palabra reservada empty. Y así en HOOP nos aseguramos que tu experiencia de programación sea la más satisfactoria posible.

Tipos de Datos :

Simples: logic ( boolean ), whole ( integer ), fract ( decimal, float ), text ( string ),  
char

Compuestos : grid ( matrix ), chain ( linked list )